

MODULE · LKR-100

Looker, end to end.

A self-paced module for the Google Cloud **Associate Data Practitioner** — visualize data, build and share dashboards, manipulate LookML, and choose between Looker and Looker Studio.

DURATION

~ 90 minutes · 60 slides

AUDIENCE

Aspiring data practitioners

PREREQ

Basic SQL · spreadsheets

OUTCOME

Build, modify and share Looker dashboards

ABOUT THIS MODULE

Built for the practitioner who reads a dashboard **and** ships one.

If you can write a SELECT and you've opened a BI tool before, you're ready. By the last slide you will be able to build, modify and share a Looker dashboard and read a LookML file confidently enough to make small changes yourself.

01 • WHO

Data analysts and engineers preparing for the Associate Data Practitioner exam.

You write SQL, you've used a BI tool. You want to understand how Looker fits Google's data stack.

02 • WHY

Looker is the BI piece of the certification — and the part most candidates underprepare for.

Dashboards, LookML, and Looker vs Looker Studio together account for a meaningful slice of the exam.

03 • HOW

Self-explanatory slides with diagrams, mock UI and real LookML snippets.

No video, no setup, no sign-in. Read, click through, and refer back to the cheat sheet at the end.

LEARNING OBJECTIVES

By the end you can **do**, not just describe.

Mapped one-to-one with the exam's Looker pointers — every objective is a verb you should be able to act on inside Looker the next time you open it.

OBJECTIVE 1

Create, modify and share dashboards.

Build a dashboard from Explores, add tiles, configure filters, schedule deliveries and manage access through folders.

OBJECTIVE 2

Manipulate simple LookML parameters.

Read a model file, identify dimensions, measures and joins, and edit common parameters such as **LABEL** and **SQL**.

OBJECTIVE 3

Compare Looker and Looker Studio.

Pick the right tool for an analytics use case based on data modeling, governance, scale and connectivity.

OBJECTIVE 4

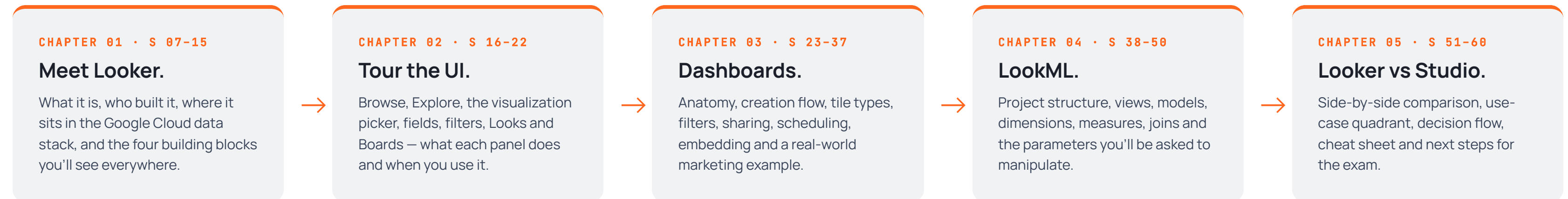
Visualize data for a business question.

Choose appropriate chart types, structure dashboard layouts and write captions a stakeholder can act on.

MODULE MAP

Five chapters, sixty slides.

Each chapter is a self-contained block — read in order on a first pass, then jump back into individual chapters using the slide rail.



WHERE IT FITS

Looker is the **presentation** layer on top of your warehouse.

It connects to BigQuery, Snowflake, Postgres, Spanner and 60+ other databases — the data never leaves them. Models are written once in LookML and reused by every chart, dashboard and embedded experience.

UX

Consumers — people who look at data.

Executives, marketers, support agents, customers in embedded apps. They open dashboards, not databases.



BI

Looker — semantic + visualization layer.

LookML models · Explores · Looks · Dashboards · Embeds · Scheduled deliveries · Looker API. The subject of this module.



SQL

Warehouse — BigQuery, Snowflake, Postgres, Spanner ...

Where the data physically lives. Looker writes SQL against it on every dashboard render — no extracts, no cubes.



ETL

Ingestion & transformation — Dataflow, Dataform, dbt, Datastream.

Pipelines that land raw data and shape it into modeled tables ready for the warehouse to query.

CONVENTIONS

How to read this deck.

A few visual cues that recur on every slide — once you can read them, you can skim the deck and still walk away with the right model in your head.

CODE BLOCKS

Real LookML, not pseudocode.

```
dimension: order_amount {  
  type: number  
  sql: ${TABLE}.amount ;;  
}
```

Highlighted lines mark what we're focusing on. Comments use the **GREY ITALIC** style.

MOCK UI

Numbered annotations.

- 1 Each circled number maps to the text on the same slide.

UI screenshots are stylized illustrations — close to Looker but stripped to the essentials so the diagram reads at a glance.

CALLOUTS

Three flavours to scan for.

EXAM turns up directly on the Associate Data Practitioner test.

TIP a shortcut, a gotcha, or a sensible default.

RECAP one-sentence summary at the end of a chapter.

Use the badges to triage on a re-read: read the EXAM callouts last before sitting the test.

CHAPTER 01 • SLIDES 07-15

Meet Looker.

What it is, where it came from, and the four building blocks every Looker user works with – Models, Explores, Looks, and Dashboards.

IN THIS CHAPTER

01 Meet Looker

02 Tour the UI

03 Dashboards

04 LookML & parameters

05 Looker vs Looker Studio

DEFINITION

Looker is a **governed** BI platform built on a reusable semantic model.

Looker is Google Cloud's enterprise business-intelligence product. Instead of every analyst writing their own SQL against raw tables, a team defines the business logic once — in a language called LookML — and everyone builds dashboards on top of that single, version-controlled model.

IN ONE SENTENCE

A modeling layer plus a visualization layer, with no extract.

Looker translates user clicks into SQL, sends it to your warehouse, and renders the results — live, every time.

WHAT IT ISN'T

Not a database. Not an ETL tool.

Looker doesn't store your data and doesn't move it around. It models, queries and visualizes data that already lives in your warehouse.

WHERE YOU'LL MEET IT

Inside Google Cloud Console, at looker.com or embedded in your own product.

Original Looker (Original) and Looker (Google Cloud core) share the same engine. The exam treats them together.

A SHORT HISTORY

From a Santa Cruz start-up to the Google Cloud data platform.

Why this matters: Looker's vocabulary (Explores, Looks, LookML) predates Google. Once Google acquired the company, the original product was kept and a sister product — Looker Studio — was rebranded next to it.

2012

FOUNDED

Looker founded in Santa Cruz, California.

Ships LookML, an in-database semantic-modeling language designed for SQL-native teams.

2019

ACQUIRED

Google acquires Looker for \$2.6 B.

Looker becomes Google Cloud's official BI product, tightly integrated with BigQuery.

2022

REBRAND

Google Data Studio renamed Looker Studio.

Two products under one brand: **Looker** (enterprise, modeled) and **Looker Studio** (free, self-serve).

2024

AI ERA

Gemini in Looker becomes generally available.

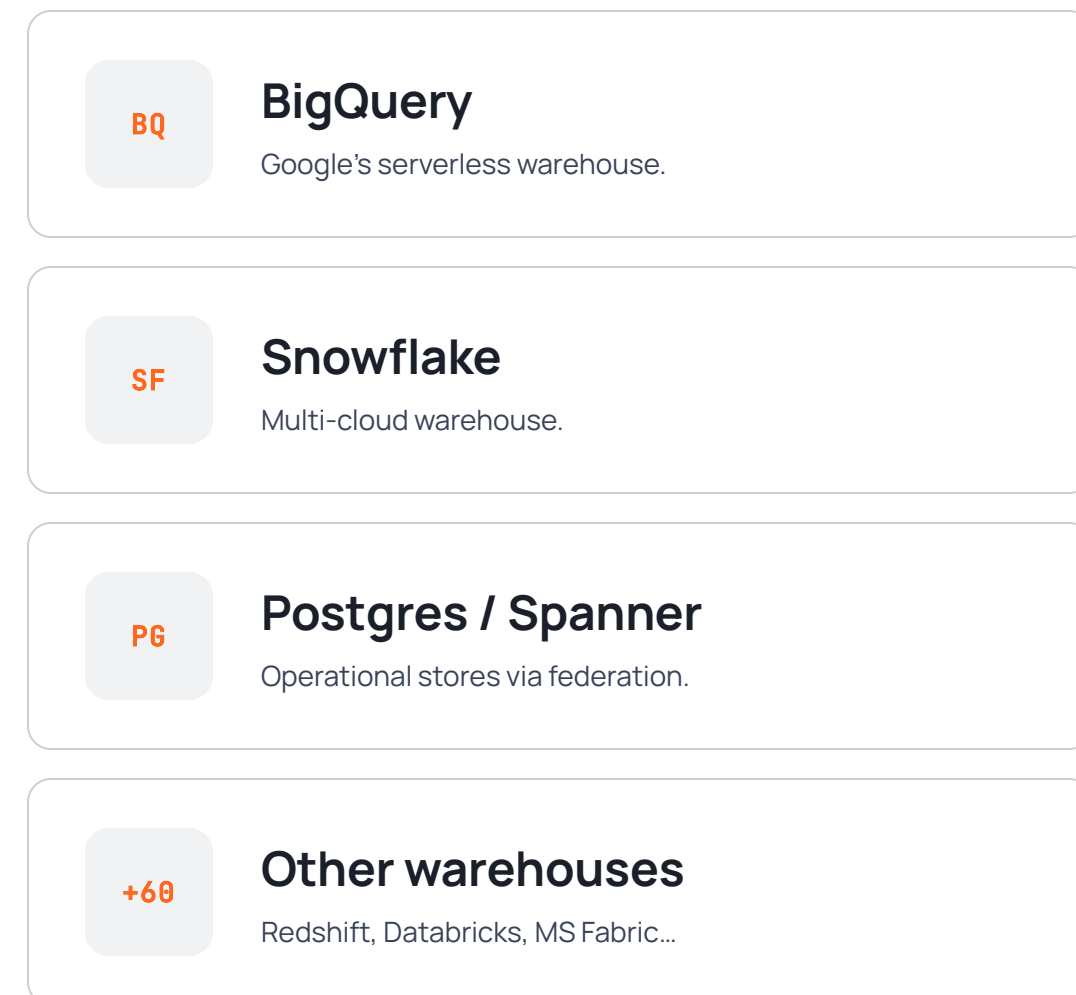
Natural-language analytics, dashboard summarization, and LookML co-authoring — all on top of the same model.

THE BIG IDEA

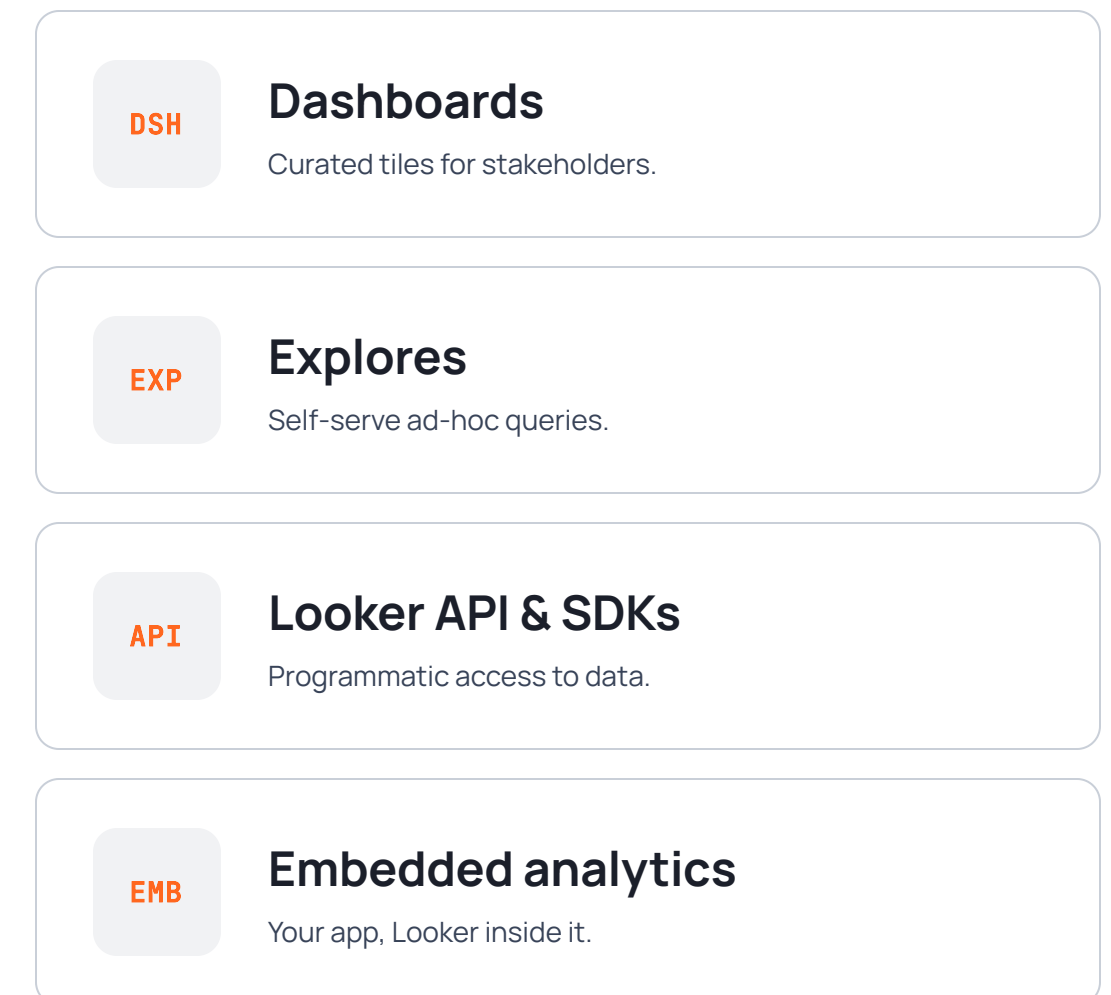
Define the business once. Re-use it **everywhere**.

Looker's superpower is a shared semantic layer. Definitions like "active user", "monthly recurring revenue" or "gross margin" are written in LookML once. Every dashboard, schedule, embed and Looker API call uses the same definition — so two analysts can't accidentally disagree by 7%.

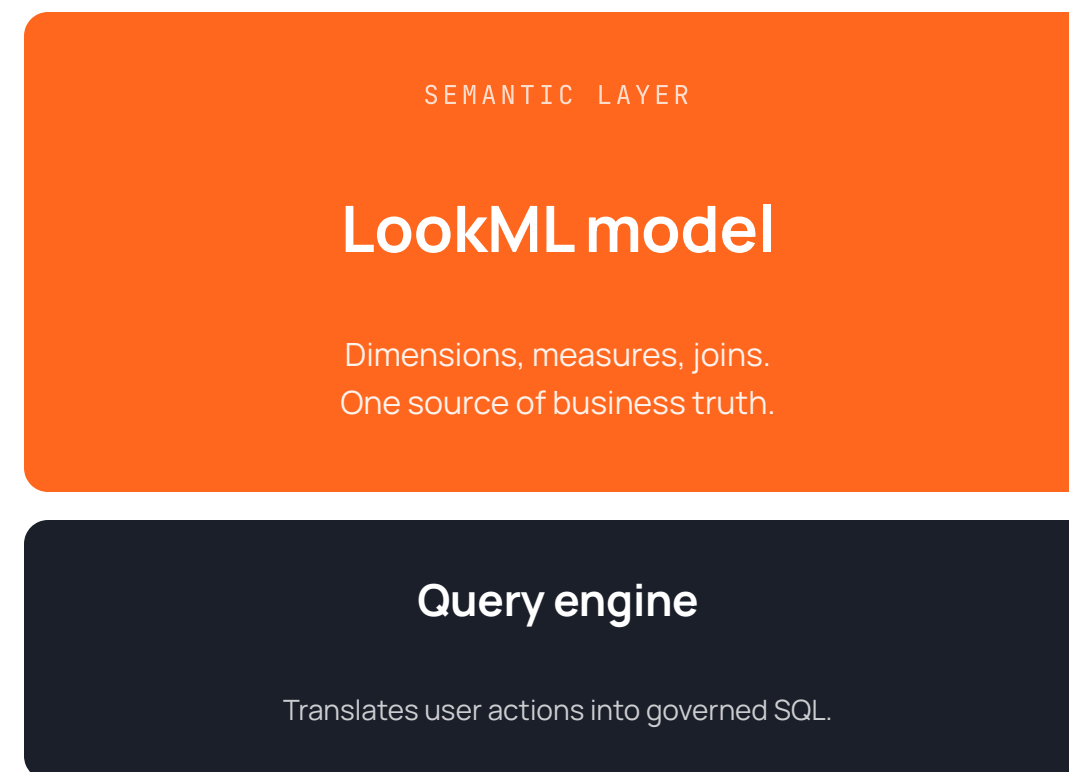
WAREHOUSES



CONSUMERS



LOOKER



WHY THIS MATTERS

A semantic layer is the difference between **one** right answer and four wrong ones.

Most BI tools let any analyst write their own SQL against the warehouse. That works at five people. At fifty, you have five definitions of "active user". Looker fixes this by putting business logic upstream of dashboards — every chart calls the same modeled definition.

WITHOUT A SEMANTIC LAYER

Five definitions of "active user", four of them wrong.

- Every analyst writes their own SQL.
- Two dashboards quietly disagree by 7%.
- Logic copy-pasted into 20 places.
- One schema change breaks reports invisibly.

WITH LOOKER'S LOOKML MODEL

One definition, version-controlled, used everywhere.

- Business logic lives in LookML, in Git.
- Every chart calls the same measure.
- Change once → all dashboards update.
- Pull requests review business logic the way they review code.

THE FOUR NOUNS

Everything in Looker is one of **four things**.

Learn these four words and the rest of the product makes sense. They build on each other — every Dashboard is made of Looks, every Look is built from an Explore, every Explore is defined in a Model.

01 • THE MAP

Model

A LookML file. Tables become Views; Views combine into Explores. Written by a Looker Developer.

Sometimes called the "semantic model" — the source of every metric.

02 • THE QUERY

Explore

A self-serve query surface. Pick dimensions and measures from the field picker; Looker generates SQL.

"Order analytics"; "Web sessions"; "Support tickets" — usually one Explore per topic.

03 • THE CHART

Look

A saved query plus its visualization. A reusable chart you can drop into multiple dashboards or schedule on its own.

Looks behave like single-tile dashboards — one chart, one URL.

04 • THE PAGE

Dashboard

A collection of tiles (Looks or Explore-based queries) arranged on a canvas with shared filters.

What stakeholders actually look at. The unit you share, schedule and embed.

EXAM

Memorise the chain — **Model → Explore → Look → Dashboard** . The certification's wording will sound natural once it does.

WHO DOES WHAT

Three personas, one platform.

Looker's permissions model is rich, but on the certification you'll see three archetypes. Each persona spends most of their time in a different surface of the product.

PERSONA 01

Looker Developer

Works in the LookML IDE. Defines Views, Explores, Models.
Commits LookML to Git, runs the validator, deploys releases.

DEVELOP DEPLOY_TO_PRODUCTION MANAGE_MODELS

PERSONA 02

Analyst (Explorer / Author)

Builds Looks and Dashboards on top of existing Explores.
Drag-drops fields, picks visualizations, configures filters and schedules deliveries.

EXPLORE SAVE_LOOKS CREATE_DASHBOARDS
SCHEDULE

PERSONA 03

Viewer (Consumer)

Opens dashboards, applies filters, downloads PDF/CSV,
subscribes to a daily email. Doesn't write SQL or LookML.

SEE_LOOKS SEE_DASHBOARDS DOWNLOAD

TIP

The exam often frames a scenario as a persona — "an analyst needs to..." usually means an Explore-level action, "a developer needs to..." usually means a LookML change.

WHERE LOOKER RUNS

Two products, three deployment shapes,
one billing surface.

You don't need to memorize SKUs for the Associate exam — you do need to recognise the deployment names when they show up in a question.

ASPECT	Looker (Google Cloud core)	Looker (Original)	Looker Studio
HOSTING	Provisioned via Google Cloud Console.	Hosted instance, managed by Looker.	Multi-tenant SaaS — no instance.
MODELING	LookML, full feature set.	LookML, full feature set.	No LookML. Per-report data sources.
IDENTITY	Google IAM or SAML.	SAML, OIDC, Google.	Google account.
COST SHAPE	Per-user or capacity billing.	Per-user or capacity billing.	Free; Pro tier for advanced features.
EMBEDDING	Full SDK, signed URLs.	Full SDK, signed URLs.	Public links, embeds with limits.
BEST FOR	Enterprise BI on GCP.	Existing Looker customers.	Ad-hoc reports, marketing analytics.

CHAPTER 01 • RECAP

04 words

Model • Explore • Look • Dashboard.

Looker is a governed BI platform built on a LookML semantic layer. Developers write the model, analysts build Looks and Dashboards on Explores, viewers consume them. Live SQL against your warehouse — never an extract. One definition of every metric, used everywhere.

RECAP

CHAPTER 02 • SLIDES 16-22

Tour the UI.

A walk through the four screens you'll spend most of your time on — Browse, Explore, Look and Dashboard — and how they connect.

IN THIS CHAPTER

01 Meet Looker

02 Tour the UI

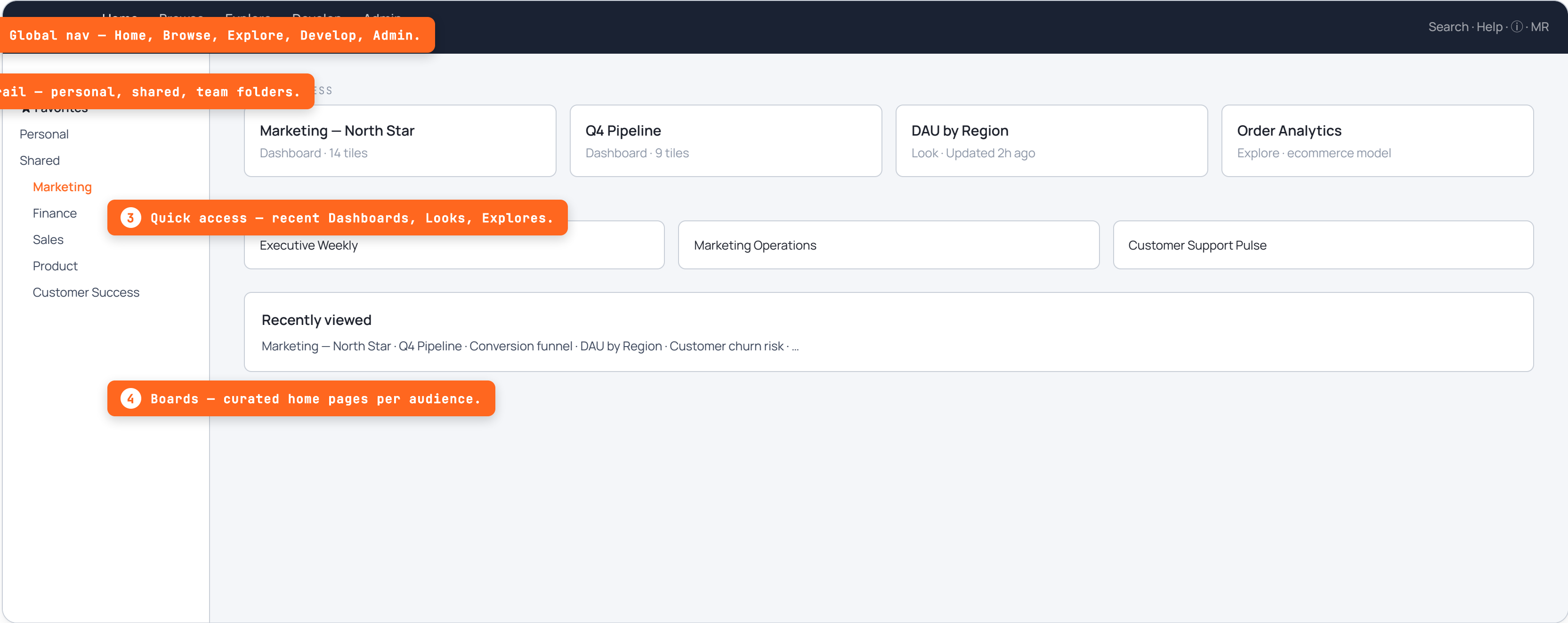
03 Dashboards

04 LookML & parameters

05 Looker vs Looker Studio

THE LANDING PAGE

Looker Home – where every session starts.



BROWSE

Folders, content, search.

Browse is Looker’s “Finder” — every Look, Dashboard and Board lives in a folder, and folders carry permissions. Where a piece of content sits decides who can open it.

acme.cloud.looker.com/browse

ALL FOLDERS

★ Favorites

Personal

▼ Shared · Marketing

↳ Acquisition

↳ Retention

↳ Brand

Sales

Finance

Product

Dashboards (8) Looks (24) Boards (2)

TITLE	FOLDER	OWNER	MODIFIED
Marketing — North Star	Marketing	Priya A.	2h ago
Q4 Pipeline	Marketing	Daniel R.	Yesterday
Email campaign perf	Marketing/Acquisition	Maya K.	3 days
Channel attribution	Marketing/Acquisition	Maya K.	1 week
Churn cohorts	Marketing/Retention	Sam L.	1 week

FOLDER HIERARCHY

Three layers, two permission models.

PERSONAL

Your private sandbox. Only you can see what’s inside.

SHARED

Team folders, governed by Group permissions.

EMBED

Reserved for embedded apps; never shown to end users.

WHAT YOU CAN DO HERE

Search, move, favourite, share, schedule, deprecate.

Search runs across titles, descriptions and tags. **Favorites** pins items to the home page; **Schedules** turns any content into an email or webhook delivery.

EXPLORE

The Explore – Looker's **query builder**.

Looker

HomeBrowseExploreDevelop

Run · Save ·

ORDER ANALYTICS

Visualization · Column · Edit · Type · Series · Y-axis

1 Field picker – every dimension

2 Visualization toolbar – switch chart type any time.

3 Filter pane & live SQL.

4 Performance footer – runtime, cache, row count.

▼ Orders

📅 Created Date

📅 Created Month ✓

📄 Status

📄 Channel

Σ Count ✓

Σ Total Revenue ✓

Σ Avg Order Value

▼ Users

📄 Country

📄 Plan Tier

Σ Distinct Count

► Products

Bar chart visualization showing monthly revenue trends with orange bars.

FILTERS

Orders.Status is Complete

Orders.Created Date is in the last 12 months

PIVOT

– none –

SORT & LIMIT

Created Month ↑ · Row limit 500

SQL

```
SELECT
  DATE_TRUNC(
    orders.created_at,
    MONTH) AS m,
  SUM(amount) AS rev
FROM orders
WHERE status='complete'
GROUP BY 1
ORDER BY 1;
```

globallogic.com

· Cache hit

19 / 60

FIELDS, FILTERS, PIVOTS

Three field types. Two ways to slice.

Every Explore exposes two field families: **dimensions** (the rows of your result) and **measures** (the aggregations). Add a **pivot** to turn a dimension into columns instead of rows. Filters can be applied **before** aggregation or **after**.

DIMENSIONS

Attributes of a single row.

String, number, date, geo, tier. They go on the GROUP BY clause.

```
 $\mathcal{A}$  users.country  
 orders.created_month  
# products.price_band
```

MEASURES

Aggregations across rows.

Counts, sums, averages, ratios. They go on the SELECT and use SUM/COUNT/AVG.

```
 $\Sigma$  orders.count  
 $\Sigma$  orders.total_revenue  
 $\Sigma$  users.distinct_count
```

FILTERS

Before vs after.

Filter=WHERE, before aggregation. **Filter on measure**=HAVING, after.

```
WHERE status = 'complete'  
WHERE date IN last 90 days  
HAVING revenue > 10000
```

EXAM If a question says "filter the underlying rows", that's a dimension filter (WHERE). "Show only categories whose total exceeds X" is a measure filter (HAVING).

VISUALIZATION PICKER

Same query, **eleven** possible charts.

When you change the visualization type in an Explore, Looker does not re-run the query — it re-shapes the result. Pick the chart that matches the business question; a list of numbers is rarely the right answer.



THREE THINGS THAT LOOK SIMILAR

Explore vs Look vs Dashboard —
when does each one come up?

A common point of confusion. They're all "places you see a chart"; the difference is whether the query is reusable and where the visualization lives.

ASPECT	Explore	Look	Dashboard
WHAT IT IS	An ad-hoc query against a model.	A saved Explore query, with its viz.	A page of tiles with shared filters.
LIVES WHERE	The Explore screen — usually unsaved.	A folder, with its own URL.	A folder, with its own URL.
REUSABLE?	Not by itself — you'd save it as a Look first.	Yes — drop a Look into many Dashboards.	Yes — share, schedule, embed.
BEST FOR	Answering a one-off question right now.	A single canonical chart you'll reuse.	A curated story for an audience.
FILTERS	Per-query.	Baked into the Look.	Dashboard-level filters apply to every tile.

CHAPTER 03 • SLIDES 23-37

Dashboards.

Create, modify and share — the longest chapter in this module and the largest slice of the exam's Looker section. Fifteen slides on the page stakeholders actually open.

IN THIS CHAPTER

- 01 Meet Looker
- 02 Tour the UI
- 03 Dashboards
- 04 LookML & parameters
- 05 Looker vs Looker Studio

DEFINITION

A dashboard is a **page**, not a chart.

A Looker dashboard is a canvas of tiles arranged for a specific audience and a specific question. Each tile is a small query — usually run on every refresh, sometimes served from cache — and tiles can share filters so the whole page moves together when a viewer slices by region, date or product.

TILES

The cells of the dashboard.

Each tile is a query plus a visualization. Tiles can be built from an existing Look (linked) or defined directly in the dashboard (free-standing).

FILTERS

Controls at the top.

Date range, region, channel, customer — applied to every tile that uses the matching field. Filters can also be locked to a specific value for a curated narrative.

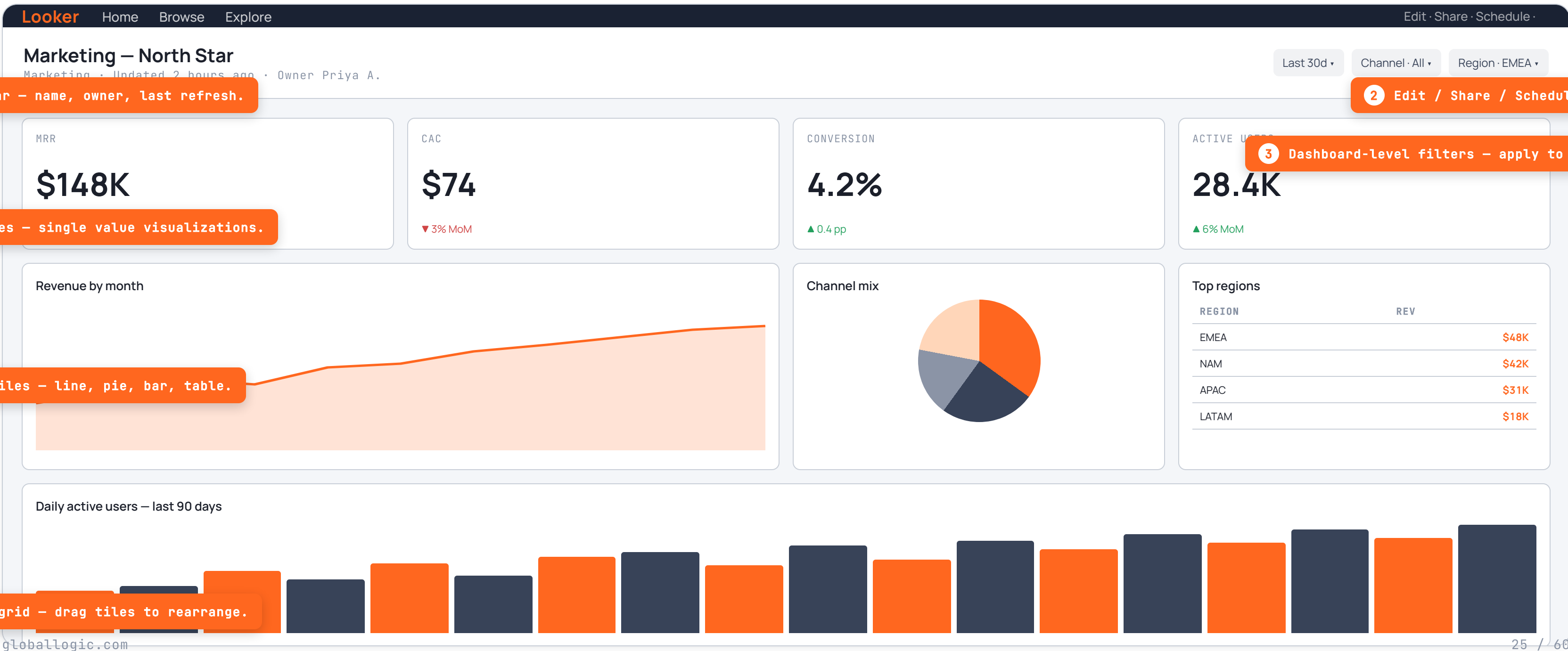
LAYOUT

Sections, rows, text.

Drag tiles around an invisible 12-column grid. Add Markdown text tiles to label sections; group tiles into expandable sections for long reports.

ANATOMY

Six parts of every Looker dashboard.



TILE TYPES

Four tile flavours — pick the right one.

Every tile is one of four kinds. Knowing the difference matters: **query tiles** rerun on every refresh, while **text and button tiles** never query the warehouse.

TYPE 01

Query tile

An ad-hoc query defined directly in the dashboard. Lives only here; if you delete the dashboard, the query goes with it.

Best for tiles unique to one dashboard.

TYPE 02

Look-linked tile

A reference to a saved Look. Update the Look and every dashboard that links to it updates too.

Best for reusable canonical charts.

TYPE 03

Text tile

Markdown — section headings, narrative captions, links to runbooks. No query, no warehouse load.

Best for “what am I looking at?” copy.

TYPE 04

Button tile

A clickable action — opens a URL, an Explore with pre-filled filters, or another dashboard.

Best for cross-linking related views.

TIP

Prefer Look-linked tiles for any chart used in more than one place — you change it once and every dashboard refreshes. Query tiles are tempting but they fragment your governance.

MATCH THE CHART TO THE QUESTION

Eleven chart types, six business questions.

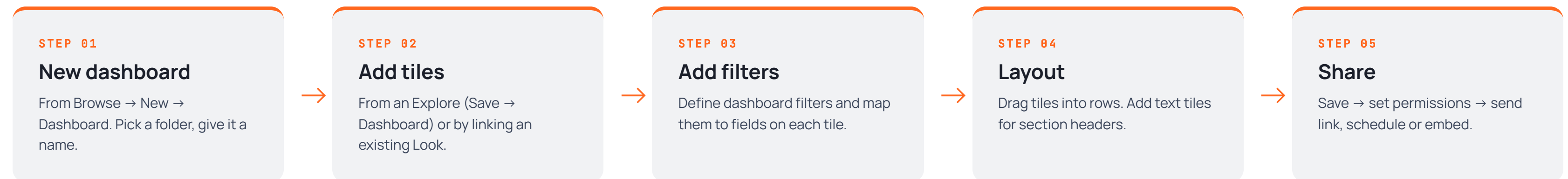
Don't memorize "use a line chart for time series" — memorize the business question. Looker's defaults are good but not always right.

BUSINESS QUESTION	First-choice viz	Why	Avoid
HOW IS X CHANGING OVER TIME?	Line / Area	Continuous x-axis emphasizes trend, not categories.	Pie, single value
HOW DO CATEGORIES COMPARE?	Column / Bar	Easy to read magnitudes at a glance; ordered top-down.	Pie with many slices
WHAT IS THE CURRENT STATE?	Single value	Headline number with optional comparison vs prior period.	Multi-axis line
ARE TWO METRICS CORRELATED?	Scatter	Each dot is an observation; pattern reveals correlation.	Two separate lines
HOW DOES THE FUNNEL NARROW?	Funnel	Drop-off between stages is the metric.	Bar chart per stage
GEO DISTRIBUTION?	Map	Spatial questions need a spatial answer.	Table of regions

CREATE A DASHBOARD

From an empty page to a shareable URL in five steps.

The exam will phrase this as "what is the correct order of steps to publish a dashboard?". Read this slide and the next two — they cover the full lifecycle.



Two creation paths. Modern **user-defined dashboards (UDD)** are built in the UI and stored in the database. **LookML dashboards** are version-controlled in Git and deployed alongside your model — best for templates and embedded apps.

ADDING TILES

Three ways to add a tile.

All three end up on the dashboard — pick based on whether you want the chart to live by itself, to be reused elsewhere, or to be a fresh ad-hoc query.

PATH A · MOST COMMON

From an Explore.

Build the chart in an Explore, click **Save → To dashboard**.
Looker copies the query into the dashboard as a query tile.

```
Explore → Save → To dashboard → "Marketing — North Star"
```

PATH B · REUSABLE

From a saved Look.

In edit mode, click **Add → Existing Look**. The tile is a live reference; updates to the Look propagate to every dashboard.

```
Dashboard edit → Add → Look → "DAU by Region"
```

PATH C · IN-PLACE

New tile from within the dashboard.

Edit mode → **Add → Visualization**. Build the query without leaving the dashboard. The tile is owned by this dashboard only.

```
Dashboard edit → Add → New tile → pick Explore → build
```


FILTER SCOPES

Three filter levels — they **stack**.

A single tile can have all three filters applied to it at once. The exam tests this — know which one wins.

1

Dashboard filter

Defined once in the title bar. Applies to every tile whose query exposes a matching field. Users can change it.

2

Tile filter

Set inside a single tile's query. Always applied, even if the dashboard filter is unset. Users can't see it.

3

Model filter (LookML)

An `ACCESS_FILTER` in the model file — limits rows by user attribute. The strongest of the three; users can't override.

WHEN THEY CONFLICT

Most restrictive wins.

Looker AND's the three together. If your model says "region = user's region", your dashboard says "region = EMEA", and your tile says "region = APAC", the tile returns **zero rows**.

```
WHERE region = 'user_region'
      AND region = 'EMEA'
      AND region = 'APAC'
— always FALSE
```

CROSS-FILTER & DRILL

Click a bar, filter the page.

Two related but distinct interactions. Cross-filtering refines other tiles in place; drilling opens a new page with the source rows.

INTERACTION 01

Cross-filter

Click a bar / slice / dot → that value becomes a temporary filter on the whole dashboard. Click again to clear.

- Enable per tile via tile options.
- Doesn't change the saved dashboard.
- Survives page refresh via URL params.

INTERACTION 02

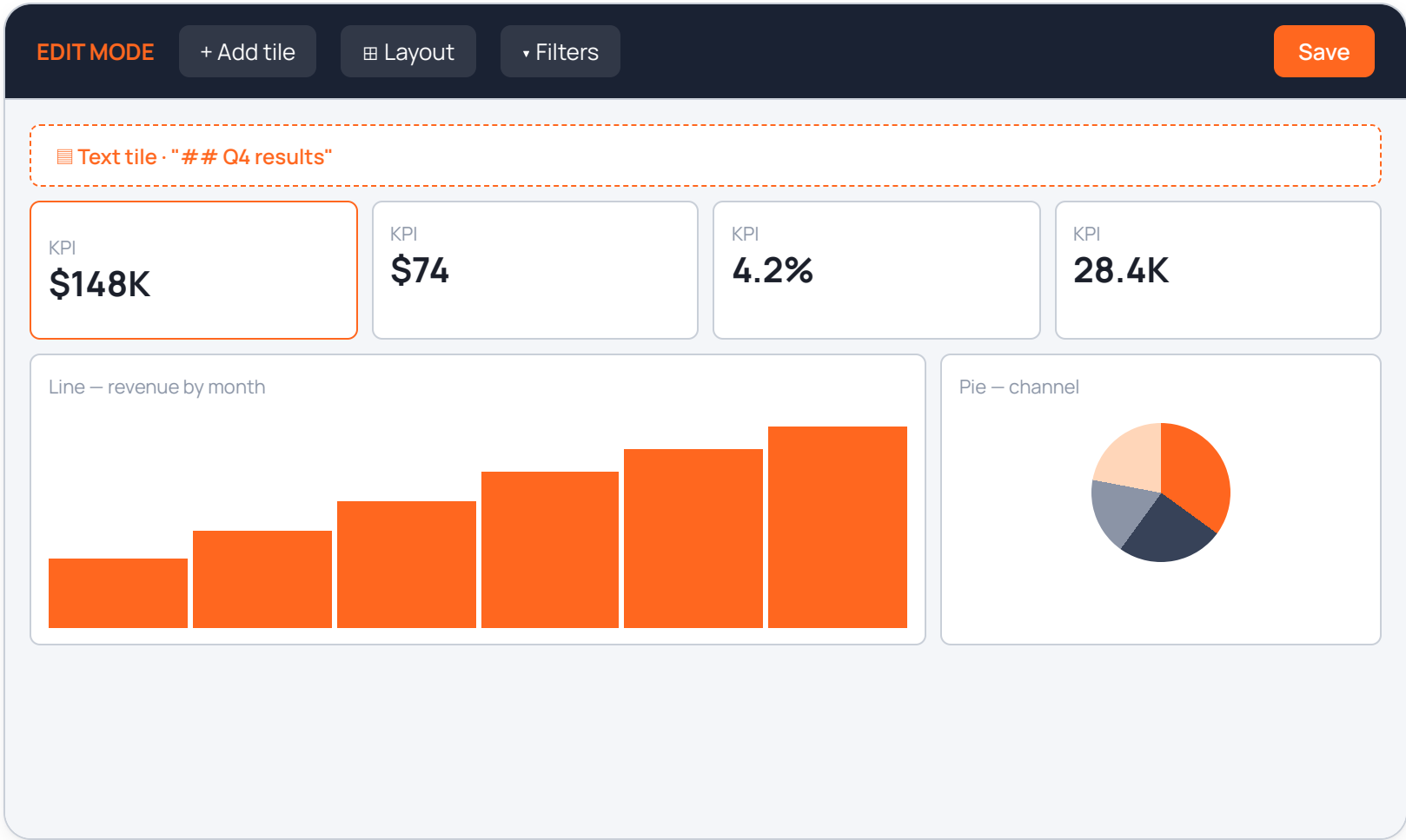
Drill

Click a value → see the underlying rows in a pop-up Explore. Define drill paths in LookML with **DRILL_FIELDS**.

```
measure: total_revenue {  
  type: sum  
  sql: ${TABLE}.amount ;;  
  drill_fields: [order_id, user.email, status, amount]  
}
```

EDIT MODE

Inside the editor — what each control does.



TILE RESIZE

Drag the bottom-right corner.

The dashboard snaps to a 12-column grid. Hold **SHIFT** to free-scale.

TILE REORDER

Drag the title bar.

Drop on another tile to swap, or onto an empty grid cell to move.

SECTION GROUPING

Markdown text tiles + collapse.

Long dashboards become sections. Each section header can be a text tile (Markdown) and tiles below can be hidden by default.

THEME & TILE OPTIONS

Edit tile → vis options.

Tile colors, axis labels, conditional formatting, totals row, value formatting (currency, percent,

MODIFYING AN EXISTING DASHBOARD

Six common edits, in the order you'll do them.

When a stakeholder asks "can you tweak this dashboard?" — these are the six things they almost always mean.

01 • FILTER

Add or change a dashboard filter.

Edit → Filters → Add. Pick the field, default value, allowed values. Map it to each tile that should respond.

02 • TILE

Add or remove a tile.

Add via Explore → Save → To dashboard; remove via tile menu → Delete.

03 • VIS

Change a chart type.

Edit tile → Visualization → pick new type. The query doesn't change; only the rendering does.

04 • TITLE

Edit copy & headings.

Click a title to rename. Edit Markdown tiles inline. Update the dashboard description for searchability.

05 • LAYOUT

Resize and rearrange.

Drag tiles into rows. Group KPIs at top, charts in the middle, tables at the bottom. Tighten spacing.

06 • PERMISSIONS

Move folder, change access.

Edit info → Move → pick folder. Folder permissions follow.

SHARING

Five ways to get a dashboard in front of someone.

"Share" in Looker means more than "send a link". Pick the mechanism that matches how the audience actually wants to consume the data.

OPTION 01 • MOST COMMON

Direct link.

Anyone with permission to the folder can open the URL. Filter values are encoded in the URL — copy after applying filters to share a specific view.

OPTION 02 • RECURRING

Schedule delivery.

Email, Slack, webhook, Cloud Storage. PDF, CSV, PNG, inline visualization. Daily, weekly, monthly, or on a custom cron.

OPTION 03 • ALERT

Threshold alert.

Notify when a KPI tile crosses a threshold. e.g. "Email me when MRR drops below \$140K."

OPTION 04 • APP

Embed.

Drop a dashboard inside your own app. Single-sign-on (private embed) or signed URL (public embed) for external customers.

OPTION 05 • EXPORT

Download.

One-off PDF, PNG or CSV for a slide deck or attachment. The user-visible "Download" button.

OPTION 06 • PROGRAMMATIC

Looker API.

Fetch dashboard data, render thumbnails, or sync content via the Looker SDK in Python / TypeScript / Go.

SCHEDULE DELIVERIES

Inside the schedule dialog.

Schedules are the most common way Looker dashboards reach non-Looker users. Every business has them; the certification expects you to know the parameters.

Schedule delivery

NAME

North Star — weekly to Exec list

DESTINATION

Email

Slack

Webhook

GCS

RECIPIENTS

exec-leads@acme.com, ceo@acme.com

FORMAT

PDF

PNG

CSV

Inline visualization

FREQUENCY

Weekly · Monday · 07:00 · America/Los_Angeles

FILTERS AT SEND TIME

Date range · Last 7 days · Region · All

SENDER

The schedule runs as **you**.

Looker queries the warehouse with your row-level permissions. If you leave the company, the schedule breaks — re-assign it to a service account or shared inbox.

FILTER OVERRIDE

Schedules can change filter values.

Send a "last 7 days" view of the dashboard every Monday — even if you've left the dashboard saved on "all time".

LIMITS

Result row caps apply.

CSV/Excel deliveries have row limits set by your admin (often 5,000–100,000). PDF renders only what fits the canvas.

FOLDERS & PERMISSIONS

Permissions live on the folder, not the dashboard.

Every Look and Dashboard inherits its access from the folder it's in. Move a dashboard, change its visibility. This is the single most common cause of "why can't Sara see this?" tickets.

Shared/

Marketing/

North Star

Q4 Pipeline

Acquisition/

Email perf

Attribution

Finance/

Revenue · CONFIDENTIAL

Sales/

Pipeline

Personal/

priya.a/

daniel.r/

// Group: marketing-team (View)

// Group: paid-team (View)

// Group: finance-team (Manage)

// Only owner

globallogic.com

THREE ACCESS LEVELS

View · Manage Access & Edit · Owner.

VIEW	See content; can't edit or delete.
MANAGE	View + edit + share + delete.
OWNER	Manage + transfer ownership.

GROUPS, NOT USERS

Assign permissions to Groups.

"finance-team" → Manage on /Finance. When Sara joins finance-team, she gets access automatically. Per-user grants don't scale.



CASE · MARKETING DASHBOARD

IN PRACTICE

One dashboard the CMO opens every Monday.

Audience. The CMO and four directs.

Top row. Four KPI tiles — MRR, CAC, conversion, active users. Single-value visualizations with prior-period comparison.

Middle. A line chart of revenue by month and a pie of channel mix, side by side.

Bottom. A daily-active-users bar chart and a top-regions table — the "what drove last week's number" detail.

Delivery. Scheduled email PDF every Monday at 07:00 with the prior week's filter pre-applied; alert if MRR drops below \$140K.

This is the dashboard whose anatomy you saw on slide 25 — and the kind the exam describes.

CHAPTER 04 • SLIDES 38-50

LookML & parameters.

The semantic model that sits underneath every Look and Dashboard. Read a model file fluently and edit the most common parameters by hand.

IN THIS CHAPTER

- 01 Meet Looker
- 02 Tour the UI
- 03 Dashboards
- 04 LookML & parameters
- 05 Looker vs Looker Studio

DEFINITION

LookML is a **YAML-like** language that describes your data — not your charts.

LookML defines what a table contains, how tables join, and what each metric means. It is then compiled to SQL on demand when a user opens an Explore or a dashboard. You write the model once; Looker writes the SQL forever.

IN ONE LINE

"Tables and joins, plus business meaning."

A LookML view describes one table. A LookML model wires views together with joins. Together they form an Explore — a query surface for analysts.

It is not: a templating language for dashboards, a query optimizer, or a database itself.

ORDERS.VIEW.LKML

LOOKML

```
view: orders {  
  sql_table_name: ecommerce.orders ;;  
  dimension: id {  
    primary_key: yes  
    type: number  
    sql: ${TABLE}.id ;;  
  }  
  dimension: status {  
    type: string  
    sql: ${TABLE}.status ;;  
  }  
  measure: total_revenue {  
    type: sum  
    sql: ${TABLE}.amount ;;  
    value_format: "$#,##0"  
  }  
}
```

LOOKML VS SQL

LookML declares. SQL runs.

LookML doesn't replace SQL — it generates it. When a user picks a dimension and a measure in an Explore, Looker assembles a fresh SQL query, sends it to the warehouse, and returns rows. The same LookML expression can produce different SQL on Postgres, BigQuery, or Snowflake.

USER PICKS FIELDS IN AN EXPLORE

LOOKML

```
// User picks:
//   • orders.created_month (dim)
//   • orders.total_revenue (measure)
//   • Filter status = 'complete'
// The model already says:
dimension_group: created {
  type: time
  timeframes: [date, month, year]
  sql: ${TABLE}.created_at ;;
}
measure: total_revenue {
  type: sum
  sql: ${TABLE}.amount ;;
}
```

LOOKER GENERATES THIS SQL

SQL

```
SELECT
  DATE_TRUNC(orders.created_at, MONTH)
    AS created_month,
  SUM(orders.amount)
    AS total_revenue
FROM ecommerce.orders AS orders
WHERE orders.status = 'complete'
GROUP BY 1
ORDER BY 1 ASC
LIMIT 500
// Sent live to BigQuery
// Cache key: hash(query+user attrs)
```

TIP When debugging unexpected numbers, click **SQL** in the Explore's right rail. The generated SQL tells you whether the model joined the way you expected, and whether your filter was applied as WHERE

PROJECT STRUCTURE

Inside a LookML project — what every file is for.

A LookML "project" is a Git repo. Three file types matter on the exam: **.view.lkml** for tables, **.model.lkml** for joins and Explores, and **.dashboard.lkml** for version-controlled dashboards.

```
ecommerce/ // the project
├── views/ // one file per table
│   ├── orders.view.lkml
│   ├── users.view.lkml
│   └── products.view.lkml
├── models/ // one per Explore set
│   └── ecommerce.model.lkml
├── dashboards/ // version-controlled UDDs
│   └── marketing.dashboard.lkml
├── manifest.lkml // project config
└── README.md
```

VIEW FILE (.VIEW.LKML)

One per database table.

Declares dimensions, measures, and the underlying `SQL_TABLE_NAME`. Reusable across multiple Explores.

MODEL FILE (.MODEL.LKML)

The Explore catalog.

Connects views via joins; each `EXPLORE:` block becomes a query surface in the Explore menu.

DASHBOARD FILE (.DASHBOARD.LKML)

Optional, but powerful.

Version-controls dashboards alongside the model. Best for embedded apps and templated

MODEL + VIEW FILE

Two files, one Explore.

Read these top-to-bottom. The view describes a table; the model describes how tables connect.

VIEWS/ORDERS.VIEW.LKML

VIEW

```
view: orders {  
  sql_table_name: ecommerce.orders ;;  
  dimension: id {  
    primary_key: yes  
    type: number  
    sql: ${TABLE}.id ;;  
  }  
  dimension: user_id {  
    type: number  
    sql: ${TABLE}.user_id ;;  
  }  
  dimension: status {  
    type: string  
    sql: ${TABLE}.status ;;  
  }  
  measure: count { type: count }  
  measure: total_revenue {  
    type: sum  
    sql: ${TABLE}.amount ;;  
  }  
}
```

MODELS/ECOMMERCE.MODEL.LKML

MODEL

```
connection: "bq_prod"  
include: "/views/*.view.lkml"  
explore: orders {  
  label: "Order Analytics"  
  description: "Revenue, AOV, orders by ..."  
  join: users {  
    type: left_outer  
    relationship: many_to_one  
    sql_on: ${orders.user_id} = ${users.id} ;;  
  }  
  join: order_items {  
    type: left_outer  
    relationship: one_to_many  
    sql_on: ${orders.id} = ${order_items.order_id} ;;  
  }  
}
```

DIMENSIONS

Dimensions describe a single row.

A dimension is an attribute of a record — a customer’s country, an order’s status, a product’s price. Dimensions appear in GROUP BY when used in queries.

THREE COMMON DIMENSION PATTERNS

LOOKML

```
// 1. Plain column passthrough
dimension: country {
  type: string
  sql: ${TABLE}.country ;;
}

// 2. Computed dimension (CASE)
dimension: price_band {
  type: string
  sql: CASE
    WHEN ${TABLE}.price < 20 THEN 'Budget'
    WHEN ${TABLE}.price < 80 THEN 'Mid'
    ELSE 'Premium'
  END ;;
}

// 3. Time dimension group
dimension_group: created {
  type: time
  timeframes: [date, week, month, year]
  sql: ${TABLE}.created_at ;;
}
```

PARAMETER CHEAT SHEET

What you'll see in every dimension.

TYPE	string · number · date · yesno · tier · zipcode · location · ...
SQL	The SQL expression Looker uses for this dimension. Ends with double semicolons.
LABEL	Display name shown in the field picker.
DESCRIPTION	Hover text for analysts. Use this generously.
HIDDEN	Hide from the field picker without deleting.
PRIMARY_KEY	Marks the row identifier. Required on at least one dim per view.
VALUE_FORMAT	Excel-style format for numbers and dates.

MEASURES

Measures aggregate across rows.

A measure is an aggregation — count, sum, average, percentile, ratio. Measures appear in the SELECT clause with a SQL aggregate function and never in GROUP BY.

FOUR MEASURE TYPES YOU'LL MEET

LOOKML

```
// 1. count — uses primary key
measure: count {
  type: count
}

// 2. sum / average / max ...
measure: total_revenue {
  type: sum
  sql: ${TABLE}.amount ;;
  value_format: "$#,##0"
}

// 3. ratio — measure of measures
measure: avg_order_value {
  type: number
  sql: ${total_revenue} / NULLIF(${count},0) ;;
  value_format: "$0.00"
}

// 4. filtered count
measure: completed_count {
  type: count
  filters: [status: "complete"]
}
```

MEASURE TYPES — SHORT LIST

The ones the exam tests.

COUNT	Number of rows. Uses the primary key.
COUNT_DISTINCT	Unique values. Requires <code>SQL: .</code>
SUM	SUM(expression).
AVERAGE	AVG(expression).
MIN / MAX	MIN / MAX.
NUMBER	Generic aggregate, often used for ratios.
DATE / TIME	For latest / earliest timestamps.

44 / 60

JOINS

Joins live in the Explore – not the view.

A view is reusable across many Explores. The Explore decides how its views relate. Two parameters do almost all the work: **TYPE** and **RELATIONSHIP**.

INSIDE AN EXPLORE: BLOCK

LOOKML

```
explore: orders {  
  join: users {  
    type: left_outer  
    relationship: many_to_one  
    sql_on: ${orders.user_id} = ${users.id} ;;  
  }  
  join: order_items {  
    type: left_outer  
    relationship: one_to_many  
    sql_on: ${orders.id} = ${order_items.order_id} ;;  
  }  
  join: products {  
    type: left_outer  
    relationship: many_to_one  
    sql_on: ${order_items.product_id} = ${products.id} ;;  
  }  
}
```

globallogic.com

TYPE – JOIN SHAPE

Which rows survive.

LEFT OUTER	All from base + matches from join. The default.
INNER	Only matched rows.
FULL OUTER	All rows from both sides.
CROSS	Cartesian product. Use carefully.

RELATIONSHIP – CARDINALITY

How Looker totals.

MANY_TO_ONE	e.g. orders → users. Safe to sum measures from the "one" side.
ONE_TO_MANY	e.g. orders → order_items. Symmetric aggregation needed. 45 / 60

LOOKML PARAMETERS

The parameters you'll actually manipulate.

The exam objective says "manipulate simple LookML parameters". These are the parameters it means — the ones you change in 90% of real LookML edits.

PARAMETER	What it does	Example
LABEL	Display name shown to analysts in the field picker.	label: "Customer Country"
DESCRIPTION	Hover help. Use to document caveats & sources.	description: "ISO-2, excl. test users"
TYPE	Data type of dimension or aggregation kind for measure.	type: number · type: sum
SQL	The SQL expression that produces the value.	sql: \${TABLE}.amount ;;
VALUE_FORMAT	Excel-style format string for display.	value_format: "\$#,##0.00"
HIDDEN	Hides from the picker without deleting.	hidden: yes
FILTERS	Built-in filter for a measure or Explore.	filters: [status: "complete"]
DRILL_FIELDS	Fields shown when a user drills into a value.	drill_fields: [id, email, amount]
globallogic.com PRIMARY_KEY	Marks the row identifier (one per view).	primary_key: yes

FILTERS IN LOOKML

Three filter-like parameters — pick the right one.

"Filter" means different things in different places. Knowing which one to use is a frequent exam question.

FILTER (ON MEASURE)

Always-on aggregation filter.

Restricts the rows fed into one measure. Invisible to the user.

```
measure: completed_orders {  
  type: count  
  filters: [status: "complete"]  
}
```

FILTER PARAMETER

User-facing filter input.

Adds a control to the Explore that doesn't filter directly — its value is referenced via `{% CONDITION %}`.

```
filter: date_range {  
  type: date  
}
```

PARAMETER

Templated user input.

A typed input passed into SQL via Liquid

`{% PARAMETER %}` — change column, period, currency at run-time.

```
parameter: grain {  
  type: unquoted  
  allowed_value: { value: "day" }  
  allowed_value: { value: "month" }  
}
```

PUTTING IT TOGETHER — LIQUID IN SQL

LIQUID

```
dimension: period {  
  type: date  
  sql: DATE_TRUNC(${TABLE}.created_at, {% parameter grain %}) ;;  
}
```

WORKED EXAMPLE 01

Modify a dimension – three small edits.

A product manager asks: "Can we relabel **Customer Tier** to **Plan Tier**, format it nicely, and hide the internal tier_id column?"

BEFORE – ORDERS.VIEW.LKML

V1

```
dimension: tier_id {
  type: number
  sql: ${TABLE}.tier_id ;;
}
dimension: customer_tier {
  type: string
  sql: ${TABLE}.customer_tier ;;
}
// – Field picker shows:
//   Tier ID
//   Customer Tier (free / paid / vip)
```

AFTER – ORDERS.VIEW.LKML

V2

```
dimension: tier_id {
  type: number
  sql: ${TABLE}.tier_id ;;
  hidden: yes
}
dimension: customer_tier {
  type: string
  label: "Plan Tier"
  description: "free · paid · vip"
  sql: INITCAP(${TABLE}.customer_tier) ;;
}
// – Field picker shows:
//   Plan Tier (Free / Paid / Vip)
```

Three changes, three parameters. Added **HIDDEN**, added **LABEL** + **DESCRIPTION**, and edited the existing **SQL** to capitalise the value. The data didn't move; only the model changed.

WORKED EXAMPLE 02

Add a new measure — average order value.

Finance wants AOV broken out by region. AOV doesn't exist as a column — it's revenue divided by order count. A measure-of-measures handles this cleanly.

ORDERS.VIEW.LKML — APPEND

LOOKML

```
// Existing — already in the view
measure: count { type: count }
measure: total_revenue {
  type: sum
  sql: ${TABLE}.amount ;;
  value_format: "$#,##0"
}

// New — append below
measure: avg_order_value {
  label: "Average Order Value"
  description: "Revenue ÷ orders, by group."
  type: number
  sql: ${total_revenue} / NULLIF(${count},0) ;;
  value_format: "$0.00"
  drill_fields: [order_id, user.region, amount]
}
```

FIVE-STEP DEPLOY

Develop → Production.

- 1 • EDIT Open the view in the LookML IDE; add the measure.
- 2 • VALIDATE Click **VALIDATE LOOKML** to check syntax + references.
- 3 • TEST Save & explore; pivot AOV by Region to verify.
- 4 • COMMIT Commit to your branch; open a pull request for review.
- 5 • DEPLOY Merge → **DEPLOY TO PRODUCTION**. Live across all dashboards.

EXAM A new measure becomes available in **every** Explore that already includes this view. You don't need to edit dashboards — they pick it up the moment you redeploy.

THE LOOKML IDE

Where developers actually work.

Looker

Home Browse Explore Develop

FILES

views

orders.view.lkml *

users.view.lkml

products.view.lkml

models

ecommerce.model.lkml

dashboards

marketing.dashboard.lkml

manifest.lkml

GIT

feature/aov-measure

2 changes uncommitted

44 sql: \${TABLE}.amount ;;

45 }

46

47 measure: avg_order_value {

48 label: "Average Order Value"

49 type: number

50 sql: \${total_revenue} / NULLIF(\${count},0) ;;

51 value_format: "\$0.00"

52 }

53

54 // drill into row-level details

Dev mode · ecommerce · feature/aov-measure

LOOKML VALIDATOR

✓ NO ERRORS. 12 files OK.

SQL RUNNER

Run query · Show generated SQL

DEPLOY

Deploy to Production

Commit & push

1 File tree – same shape as

2 Code editor with LookML syntax highlighting.

3 Validator runs on save; deploy controls below.

4 Branch & Git status – every change is a commit.

CHAPTER 05 • SLIDES 51-60

Looker vs Studio.

Two products in one Google portfolio. Pick the right tool for a use case based on modeling, governance, scale and the audience that will actually open the dashboard.

IN THIS CHAPTER

- 01 Meet Looker
- 02 Tour the UI
- 03 Dashboards
- 04 LookML & parameters
- 05 Looker vs Looker Studio

THE TWO PRODUCTS

Same family name. **Very** different jobs.

Looker (the enterprise platform) and Looker Studio (formerly Data Studio) share a brand and Google's data stack — but they serve different teams, scales and governance models. The exam wants you to know which to recommend for a given scenario.

LOOKER

Enterprise BI with a governed semantic model.

Modeled once in LookML, used everywhere. Roles, folders, row-level security, embeds, API. Paid product on Google Cloud.

Audience: Fortune 500 data teams, analyst orgs, embedded-analytics builders.

LOOKER STUDIO

Free, self-serve dashboarding for one-off reports.

Audience: Marketing, ops, individual analysts, agencies, students.

COMPARISON 01

Modeling, governance, scale.

The half of the comparison that decides whether a tool is appropriate for an organisation at all – not just for a single analyst.

ASPECT	Looker	Looker Studio
SEMANTIC LAYER	Yes – LookML. Definitions are reusable across every Look, Dashboard and embed.	No. Each report defines its own fields and calculated metrics.
VERSION CONTROL	Git-based. Every change is a commit; pull requests, branches, deploy.	None. Version history is per-report (like Google Docs).
ROW-LEVEL SECURITY	Native. User attributes & ACCESS_FILTER in LookML.	Limited. Data-source filters or BigQuery row policies.
ROLES & FOLDERS	Rich. Permission sets, groups, folder ACLs.	Drive sharing. Editor / Viewer per-report.
AUDIT & LINEAGE	Yes – admin logs, schedule history, LookML diff.	Minimal.
SCALE	Built for thousands of users + embedded apps.	Best for tens to low-hundreds of viewers.

COMPARISON 02

Connectivity, sharing, cost.

The half that determines whether a tool can answer the question you actually have today.

ASPECT	Looker	Looker Studio
DATA SOURCES	60+ warehouses via JDBC; live queries — no extracts.	1000+ connectors — Sheets, Ads, GA4, BigQuery, community.
SPREADSHEETS / FILES	Not directly — load into warehouse first.	Yes — Google Sheets, CSV upload.
SHARING	Direct link, schedule, alert, embed, API.	Drive link, schedule (email), embed.
EMBEDDING	SSO, signed URLs, full SDK, themable.	Public links, embed URL with limits.
CUSTOMISATION	LookML, custom viz JS, themes.	Drag-drop styling, community visualizations.
COST	Per-user or capacity — paid subscription.	Free. Studio Pro adds team workspaces & support.
SETUP TIME	Days-weeks (admin, model, deploy).	Minutes (connect, drag, share).

USE-CASE QUADRANT

When to pick which — at a glance.

Plot your use case on two axes: how much modeling you need vs how many people will rely on the result. Most exam scenarios land cleanly in one quadrant.

LOOKER • USE CASE

Modeled, small audience.

Internal data-team prototypes, new model validation, exploratory analytics on a governed warehouse.

LOOKER • STRONG FIT

Enterprise BI & embeds.

Customer-facing embedded analytics, executive dashboards, regulated industries, multi-team rollouts.

DECISION FLOW

Five questions – one tool.

Run a real scenario through these five questions and the answer becomes mechanical. The exam's "which tool would you recommend?" items almost always hinge on the first three.

Q1

Will more than one team rely on the same metric definitions?

Yes → consider Looker (single source of truth). No → either tool may work.

Q2

Will this dashboard be embedded into a customer-facing product?

Yes → Looker (signed URLs, SSO, SDK). Studio's embeds are limited.

Q3

Do you need row-level or column-level security?

Yes → Looker (`ACCESS_FILTER` + user attributes). Studio relies on warehouse-side policies.

Q4

Is the data in Google Sheets, a CSV or an unmodeled SaaS connector?

Yes → Looker Studio is faster, free, and natively integrated.

Q5

Is the budget zero and the audience small?

Yes → Looker Studio.

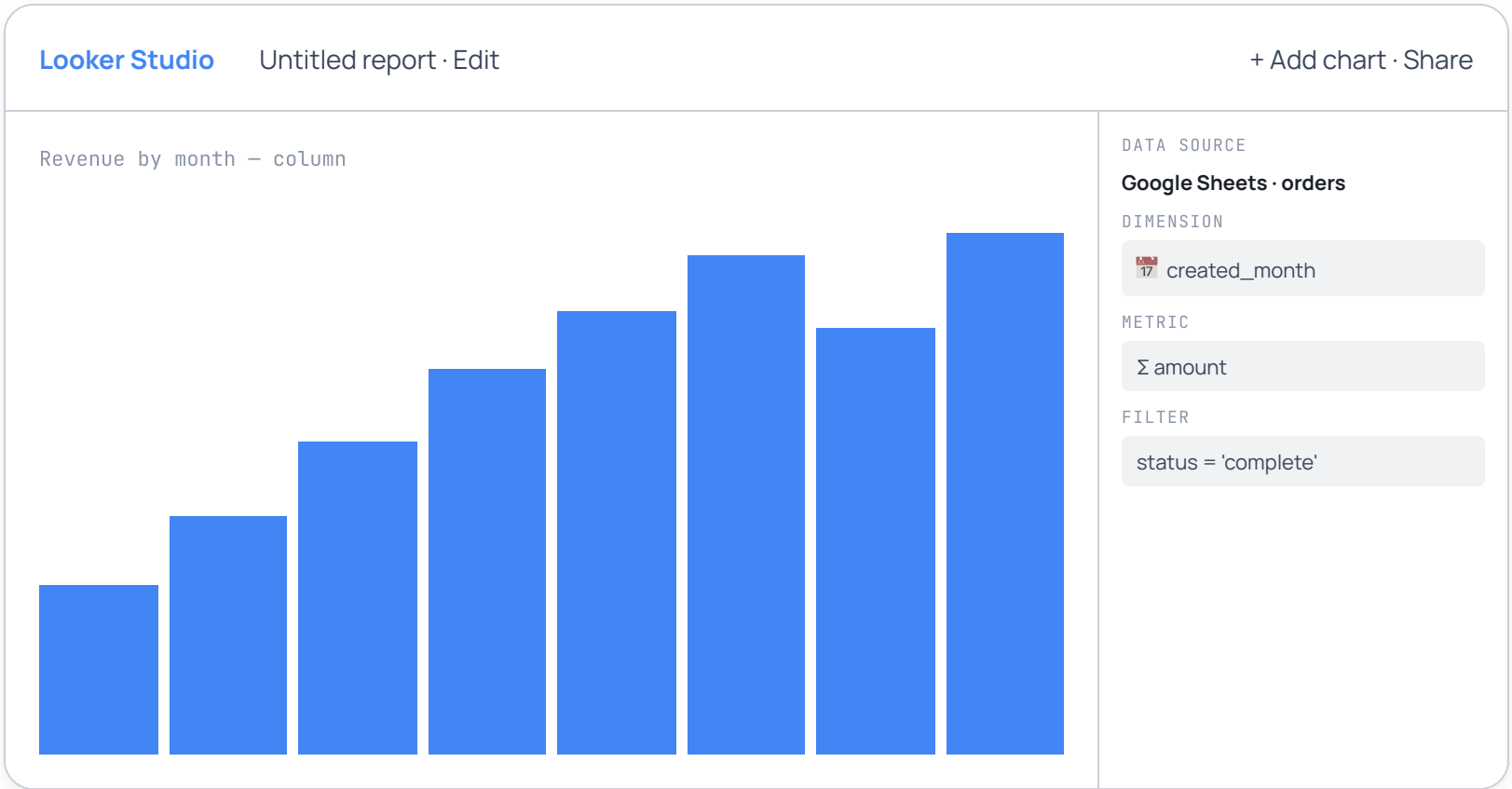
SIDE BY SIDE

Same data, two products.

The same revenue chart — built in each tool — exposes the surface differences immediately.



Looker. Field picker is the model. Same fields appear in every Explore. The chart is a query against modeled definitions.



Looker Studio. Right-rail data panel — fields are defined per-chart against the source. No shared model; the next chart starts from scratch.

CHEAT SHEET

Everything that matters,
on one slide.

Save the link. The fifteen lines below answer 80% of the Looker questions on the Associate Data Practitioner exam.

VOCABULARY	
MODEL	LookML file: views + joins + Explores.
VIEW	One table; dimensions & measures.
EXPLORE	Query surface — a joined set of views.
LOOK	Saved Explore query + visualization.
DASHBOARD	Canvas of tiles with shared filters.
TILE	One chart, text, or button on a dashboard.
FOLDER	Container with permissions.
SCHEDULE	Email/Slack/webhook delivery on a cron.

LOOKML PARAMETERS	
TYPE	number, string, date, sum, count, ...
SQL	SQL expression — ends with ;;
LABEL	Display name.
HIDDEN	yes / no — show in picker.
VALUE_FORMAT	Excel-style format string.
DRILL_FIELDS	Fields shown when drilling a value.
TYPE / RELATIONSHIP	Join shape + cardinality.
ACCESS_FILTER	Row-level security.

WHAT TO DO NEXT

After this deck, do these four things.

Reading is necessary but not sufficient. Looker rewards hands-on practice — the certification questions are written by people who assume you've used the product.

STEP 01 • FREE TRIAL

Spin up a Looker trial.

cloud.google.com → Looker → 30-day trial.
Or use the official sandbox at
try.looker.com.

STEP 02 • BUILD A DASHBOARD

Recreate the marketing example.

Four KPIs, one line chart, one pie, one bar, one table. Add a date-range filter. Schedule it to your own inbox.

STEP 03 • EDIT LOOKML

Add the AOV measure.

In the e-commerce sample project: open `orders.view`, paste the AOV measure, validate, deploy.

STEP 04 • BUILD THE SAME IN STUDIO

Then build the same thing in Looker Studio.

Connect a Google Sheet of mock data; build the four KPIs and the line chart. Notice everything you couldn't do.

OFFICIAL

cloud.google.com/looker/docs · cloud.google.com/learn/certification/data-practitioner

COMMUNITY

community.looker.com · LookML on Stack Overflow

PRACTICE

Qwiklabs Looker quests · Google Cloud Skills Boost

Open Looker. Build something **today.**

MODULE

LKR-100 · Associate Data Practitioner

AUTHOR

GlobalLogic Learning · Data Practice

VERSION

Q1 · 2026 · 60 slides

FEEDBACK

learning@globallogic.com